PLATFORM BUILT IN

lifepal

PRESENTED TO

CODAT

DATA & INSIGHTS

DATA ENGINEERING · FIRESIDE CHAT · LIFEPAL TO CODAT

Nexus

Business data that people and AI can query, and trust

Four years leading, and writing the production Python for, the data platform at Lifepal, a regulated insurance marketplace. CDC into a Delta lakehouse on Databricks, dbt models with 500+ checks, a semantic layer with one definition per metric, and an MCP server so AI agents query certified data instead of raw tables. Here it is applied to Codat's Insights platform, built on the accounting, ERP, banking and commerce data Codat connects for SMBs. Every Lifepal figure traces to a run or a query.

PRESENTED BY

R. Firman Insan Muhammad (Vim)
Senior Data Engineer · London

TO

Data & Insights
Codat

ROLE

Senior Data Engineer
London hybrid / UK remote

STACK

Python · Spark · Databricks · Delta
dbt · Dagster / Airflow · semantic layer
· MCP

CONTENTS

Contents

I'd like to start with results and how I work, then go into the technical detail as far as you'd like.

PART 1	**Results and fit**	What I delivered, measured honestly; the role line by line; how I work.	03–05
PART 2	**Codat's data and its users**	What makes SMB business data hard, who the Insights platform serves, contracts, and how I handle requests.	06–10
PART 3	**Team and delivery**	How the team ships, the CI gate, and First Light: my playbook for urgent demand and reduced capacity.	11–15
PART 4	**Platform and governance**	Platform shape, layers, tenant isolation, sensitive data, lineage, quality monitoring, retention and erasure.	16–22
PART 5	**Data flow and modelling**	Source contracts, Dagster assets, ingestion, safe replay, time travel, one canonical model across platforms, SCD2, incremental facts, quality at the boundary.	23–31
PART 6	**Semantic layer, MCP and AI**	One definition per metric, MCP tools for agents, evaluation, an on-call agent, AI in daily work, and cost.	32–37
CLOSING	**First 90 days and summary**	How I'd join the team, the questions I'd ask, and the evidence on one page.	38–39

How to read it. Parts 1–3 are about outcomes, people and delivery. Parts 4–6 are the technical evidence behind them, to go as deep as you like. Every Lifepal figure traces to a run or a query.

Limits are marked in amber where the mechanism differs from what I ran in production, or where my depth is working-level. Anything about Codat is my reading of public material, shown as a proposal.

Working code on synthetic data, and the live site: github.com/rfim/nexus-codat · rfim.github.io/nexus-codat

Measured outcomes

The five numbers I'm proudest of from Lifepal. I've put the measurement basis next to each one, so you can ask me how I got there.

6h → 90m
Nightly batch wall-clock. Median of 30 runs either side of the change: 4.5 hours removed.

99.7%
Reduction in reporting latency: 24 h → under 5 min, on Kafka + Debezium CDC.

$11K
Monthly platform cost against a $19K projection. Held for four consecutive quarters.

−60%
Ad-hoc requests, 90-day trailing count 214 → 86. Data-trust escalations fell by the same share.

512
Automated quality assertions across 47 sources in CI/CD. Sustained pass rate ≥ 0.98.

Attribution of the batch-window result

- **Materialisation, about 55%.** Incremental merge replaced full rebuilds; each night recomputes the last 72 hours, not four years. The core fact went from 2 h 14 min to 11 min (page 30).
- **Compute isolation, about 30%.** Ingest, transform, BI and ML each on their own compute policy, so none queues behind another. Measured as the fall in mean queue time.
- **Spark tuning and contracts at ingestion, about 15%.** Pruning, AQE, broadcast joins, skew handling, and rejecting malformed payloads before they land.

Why the basis matters more than the number

A batch-window figure with no run count behind it is a claim. Thirty runs either side of one change is a measurement you can reproduce from the job history.

The 72-hour lookback is the p99.5 of observed late-arrival delay: it comes from the data. The cost figure is a four-quarter average, so one good month doesn't flatter it.

> **On scale.** Lifepal is a mid-size marketplace: hundreds of millions of records a day under a $19K-a-month ceiling. Codat's volume and tenant count differ; the methods (contracts, layer rules, replay safety, cost attribution) are the same.

How my experience maps to the role

I went through the Senior Data Engineer (Data & Insights) description line by line. Here is where I've done each part, and where in this pack we can talk about it.

WHAT THE ROLE ASKS FOR	WHAT I HAVE DONE, AT LIFEPAL UNLESS STATED	PAGES
Production code most days, mainly Python	Wrote the platform's production Python: a config-driven ingestion framework, CDC merge logic and FastAPI services, tested with pytest. Not wiring tools together.	23–27
Pipelines from scratch	Built Nexus from the ground up: PostgreSQL, APIs and partner files → Kafka + Debezium, DMS, Airflow → Delta Bronze/Silver/Gold on Databricks → dbt.	16 · 23–31
SQL, Spark, Databricks / Delta, orchestration, dbt	All in production. Airflow in depth; 140 jobs generated from contracts. Dagster's asset model maps onto how I already treat tables as products.	24 · 27–30
Own projects end to end	Batch → lakehouse migration: options, prototype, ADRs, build, rollout and on-call. Reporting 24 h → under 5 min.	03 · 11
Testing, observability, data quality	512 assertions across 47 sources in CI, freshness SLOs, 2σ volume holds, quarantine with the failing rule attached, runbooks.	21 · 31 · 35
Semantic layer and MCP	In production: canonical metrics agreed with Finance, Marketing and Product in a dbt semantic layer over Kimball stars; an MCP server for governed agent queries; a RAG copilot (1,000+ outcomes a day).	32–34
Agents that diagnose and fix pipelines	Proposal: a read-only on-call agent that drafts a diagnosis and a fix as a PR, measured against time to diagnose.	35
CI/CD, containers, cloud	GitHub Actions, Docker, Terraform-managed AWS (S3, Glue, Lambda, DMS, ECS). Kubernetes at Deloitte. Zero manual production changes.	12
Set technical direction and standards	Set the platform's direction with written reasoning. Led 3–5 engineers: two promoted to Senior, onboarding 4 weeks → 2. Taught 100+ learners at Hacktiv8.	11–15
AI tools day to day	Claude Code and Cursor daily for code and tests; AI for research and domain learning; prototypes built to test an architecture before committing.	36

How I work: build and serve

If I joined the team, this is what my weeks would look like. Pick any moment and we can talk through what I'd do and why.

BUILD

The pipelines and the platform

1. **Design it.** A new Insights product arrives. I lead the design one decision at a time and write it up as an ADR, with a prototype where the risk is real.
2. **Ship it.** From the first question to production through the CI gate: tests, replay check, blast radius, metric diff.
3. **Raise the bar.** A teammate's first pull request. I review it so they learn, without blocking them.
4. **Model it.** Three accounting platforms describe the same invoice three ways. One canonical model, every version kept, one Kimball view on top.

SERVE

The people and systems that use the data

5. **Work with the business.** A week of requests from product, customer-facing teams, data science and risk.
6. **Keep it running.** Monday, 07:02, on call. A customer's cash-flow insight moved overnight. I find out why before 09:00.
7. **Protect tenants.** Each company's data is visible only to the customer it was shared with, for agents too.
8. **Let them ask.** Three reports, three numbers for one metric. I agree one definition, publish it in the semantic layer, and an agent answers questions from it with a citation.

The foundation. Every page of rfim.github.io/nexus-codat reads the same run of the PySpark code in this repo, on synthetic SMB accounting, ERP, banking and commerce data, so the numbers agree from page to page. Start with **Enable MCP**: the protocol exchange, animated step by step.

What makes SMB business data hard

Codat connects SMB accounting, ERP, banking and commerce data for the banks, lenders and platforms that serve them. From the outside, these are the problems I'd expect the Insights platform to solve.

01 Many sources, one meaning

Each accounting, ERP, banking and commerce platform models an invoice, a payment or a chart of accounts differently. Insights need one conformed definition on top.

02 History gets rewritten

Books are edited, backdated and closed late. A pipeline must take the newest version of a record and survive replays, or a lender sees two numbers for the same month.

03 Per-company trust

Each SMB consents to share its data with a specific customer. Isolation by company, access by purpose and an audit trail belong in the data model.

04 Uneven quality

Missing categories, duplicated bank lines, mixed currencies, abandoned ledgers. Quality has to be measured per source and per company, then held at the boundary.

05 Decisions depend on it

Credit and cash-flow insights feed real lending decisions. Every metric needs an owner, a grain, a freshness promise and lineage you can show.

06 AI is a new consumer

Agents will ask questions of this data. They need certified metrics and governed tools, not raw tables, or they'll answer confidently and wrongly.

What Codat connects, as listed on codat.io: accounting and ERP (Xero, QuickBooks Online, Sage, NetSuite, Dynamics 365 Business Central, FreeAgent), banking (open banking feeds, partly via Plaid and TrueLayer) and commerce (Shopify, Stripe, Square, Amazon Seller): more than 350,000 live connections, as the site states.

These are hypotheses from public material. I'd like to hear which match reality and which I've got wrong.

Who the Insights platform serves

Each group asks a different question, so I'd work with each a little differently. The Codat column is my assumption, to check.

GROUP	WHAT THEY ASK FOR	WHAT I BUILD FOR THEM	HOW I WORK WITH THEM
Insights product team	Products customers can rely on, shipped quickly.	Certified data products with contracts, SLOs and versioned metrics.	Product owns the definition; I own the pipeline. Definitions agreed in writing before code.
Customers: lenders, banks, platforms	The same number every time they ask, fresh enough to decide on.	Stable APIs and exports over gold, with published freshness and change notes.	Breaking changes are versioned; restatements are explainable from history.
Data science and ML	Features without leakage, reproducible training sets.	Point-in-time feature views and as-of joins; snapshots by Delta version.	Shared conventions on keys and time; features reviewed like any other model.
Platform and API engineering	Clean events and schemas they can evolve safely.	Source contracts, schema-change checks, blast radius on the PR.	Embedded early; a new field is reviewed before it reaches a product.
Risk, security, compliance	Evidence: who saw what, why, and for how long it's kept.	Tenant isolation, masks, retention schedule, erasure runs, access logs.	Access is a business decision recorded in a governed table; engineering makes it a reviewed run.
AI systems and agents	Tools that answer correctly and stay inside permissions.	Semantic layer + MCP with citations, refusals and an eval set.	Treated as a consumer with its own contract and tests.

The closest parallel from Lifepal: B2B. Lifepal also connected SMBs to insurers: a small business shared its company and team data once, with consent, and several insurers, each with its own format, quoted on it. Collect once, map by contract, scope by consent, decide on one agreed number: the same shape as an SMB connecting its accounting, ERP, bank and commerce data to a lender through Codat.

Data contracts. Every conversation with a group ends in a data contract: definition, grain, freshness, quality rules and access, written as YAML and turned into tests (next page). They own the definition; I own the platform that keeps it true.

Agreements become data contracts

Something I've always done: every agreement ends as a YAML contract, and the tests come straight from it. The example is illustrative, in Codat's language.

CONTRACTS/PRODUCTS/INSIGHTS_CASH_FLOW_MONTHLY.YML — YAML

```yaml
product:    insights_cash_flow_monthly
team:       Insights
owner:      insights-product-lead     # signs off the definition
engineer:   vim                       # owns the pipeline
definition: >
  Net operating cash flow = bank inflows minus outflows for the month,
  excluding transfers between the company's own accounts and loan drawdowns.
grain:      [customer_id, company_id, currency, month]
freshness:
  column: synced_at
  max_lag_hours: 24                  # after the company's latest sync
columns:
  - name: company_id
    tests: [not_null]
  - name: currency
    tests: [not_null, {accepted_values: [GBP, EUR, USD, AUD, CAD]}]
  - name: net_operating_cash_flow
    tests: [not_null]
reconcile:
  against: silver.bank_transaction
  match_on: [company_id, month]
  tolerance: 0.01
access:
  read: [grp_insights_api, grp_risk_analysts]
  tenant_column: customer_id          # row filter, page 18
change:
  breaking: owner_approval            # blast radius posted on the PR
```

WHAT THE YAML TURNS INTO

- **Tests.** Column tests, grain uniqueness and reconciliation against silver, run in CI and on every load.
- **A freshness SLO**, with an alert routed to the named owner and reported as attainment over time.
- **Grants and a tenant filter.** Who may read the product, and the column that scopes each row to one customer.
- **Change control.** A breaking change needs the owner's approval on the pull request.

WHY TEAMS ACCEPT IT

- They read a short YAML file. The diff shows who agreed what, and when. A disagreement becomes a pull request with options, so there is a record.

> Same mechanism end to end. Source contracts drive ingestion (page 23); product contracts drive tests and access. That is how 512 checks reached CI at Lifepal.

See the code: github.com/rfim/nexus-codat/src/nexus/contracts.py

How I handle requests

Whoever is asking, I follow the same few rules. The main one is no surprises.

When a request arrives

- **Start from the decision.** What will you do differently with this number?
- **Check the certified metrics first.** Most requests are already answered by one.
- **New definitions go into the semantic layer first**, with a named owner, and into the report or API second.
- **Explain trade-offs in their terms:** freshness, cost and risk.

When the answer is no, or not yet

- **A reason and an alternative, every time.** "Not every 15 minutes, but six-hourly meets your SLA at a fraction of the cost."
- **Definition disputes go to the data owner**, with the options written down.
- **Bad news early and in writing.** A blameless incident write-up, published where the business can read it, the same week.
- **Cost gets a name on it.** Publishing spend per team changed behaviour with no policy change at all.

PARTNERS WHO KEEP IT SAFE	WHAT THEY NEED	HOW I WORK WITH THEM	EVIDENCE, AT LIFEPAL
Architecture and platform	Solutions inside agreed standards	Significant decisions written up as ADRs before the build; standards enforced as CI checks.	Zero manual production changes.
Security, risk, privacy	Evidence they can check	Access recorded in a governed table; retention and erasure calls are theirs, engineering makes each one a reviewed run.	Aligned with GDPR and audit requirements.
Engineering manager, product	Predictable delivery, visible trade-offs	One weekly forum for incidents, delivery and cost together. Risks listed on the roadmap up front.	$11K against $19K for four quarters.

Result at Lifepal. Ad-hoc requests fell from 214 to 86, and data-trust escalations fell by 60%.

A certified metric catalogue for Insights

An idea to discuss: the metrics lenders and platforms ask about most, each defined once, with its grain, source and the trap that makes it hard. Names and definitions are illustrative.

METRIC	QUESTION IT ANSWERS	GRAIN	SOURCE DOMAINS	THE TRAP
Net operating cash flow	Is the business generating cash?	company × month	Banking	Own-account transfers and loan drawdowns look like inflows
Revenue and growth	Is revenue growing, and steadily?	company × month	Accounting, commerce	Credit notes, backdated invoices, refunds booked months later
Days sales outstanding	How quickly do customers pay?	company × month	Accounting (invoices, payments)	Partial payments, write-offs, invoices voided after payment
Gross margin	Is each sale profitable?	company × month	Accounting (P&L mapping)	Every chart of accounts differs; mapping changes over time (SCD2)
Customer concentration	Does one customer dominate revenue?	company × quarter	Accounting (customers, invoices)	Same customer under several names and IDs
Balance volatility	How stable is the cash position?	company × day	Banking	Missing days, multiple accounts and currencies
Data completeness	Can this company's insights be trusted?	company × source	All, via sync metadata	A quiet ledger may be inactive, or disconnected

Why the last row matters. A completeness score per company, published next to every metric, lets a lender tell "revenue fell" apart from "data stopped arriving". It is the cheapest way to prevent a wrong decision.

To validate. I don't know which of these Codat already certifies, or how. This is where I'd start listening, not a plan to replace anything.

How the team works

How I run delivery day to day as a senior engineer who sets direction. I still write and review code every day; it keeps my priorities honest.

Operating cadence

- One weekly forum for incidents, delivery and cost together. They're the same conversation; separating them hides the trade-offs.
- Every pull request reviewed within one working day.
- Significant design decisions written up as ADRs, short and dated.

Standards as code

- Lint, types and tests run on every PR.
- Documentation is part of done: an undescribed model doesn't merge, which is how docs survive a busy quarter.
- A blameless write-up for every incident, published where the business can read it.

What it produced at Lifepal

- Two engineers promoted to Senior, on stretch work with written success criteria.
- Onboarding 4 weeks → 2: pairing from day one, standard model docs, CI that catches defects before a reviewer has to.
- Zero manual production changes.

Batch to lakehouse, as a decision record

- **Context:** fragile nightly batch, 24-hour-old reports, re-runs pushing past the window.
- **Options:** tune the batch, warehouse-only, or lakehouse with CDC. I prototyped the last one first, with AI, on real data.
- **Decision:** CDC into Delta, dbt for modelling, incremental facts, compute isolation.
- **Explained** to engineers and commercial leads in their terms: freshness, cost, risk.

Reviews that teach

On a new engineer's PR I flag the line, explain why it matters, and suggest the fix. Typical first-PR issues: a committed secret, personal data in gold, append instead of MERGE, `toPandas()` on a large table, no test. I prove the bug on data where I can, so the lesson sticks.

The CI gate a new joiner meets on day one

One deployment path. Every change, mine included, goes through the same checks.

.GITHUB/WORKFLOWS/CI.YML — GITHUB ACTIONS

```yaml
on: { pull_request: { branches: [main] }, push: { branches: [main] } }
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ruff check . && mypy src/
      - run: pytest -q                        # transforms, replay, late data
      - run: python contracts/compile.py --check  # contracts → tests
      - run: dbt build --select state:modified+ --defer --state prod/
      # posts the downstream set and its owners onto the PR
      - run: python ci/blast_radius.py --pr ${{ github.event.number }}
      # a certified metric moving > 0.5% fails the merge
      - run: python ci/metric_diff.py --threshold 0.005
      - run: python ci/agent_evals.py         # MCP question set, page 34
      - run: terraform -chdir=infra plan -input=false
  deploy:
    needs: validate
    if: github.ref == 'refs/heads/main'
    environment: prod                         # approval gate
    steps:
      - run: docker build -t nexus:${{ github.sha }} . && docker push ...
      - run: databricks bundle deploy -t prod
```

WHAT EACH STEP PROTECTS

- **Replay tests** prove a re-run changes nothing (page 27).
- **State-aware dbt build** runs only what changed, deferring the rest to production state, so CI stays fast.
- **Blast radius** answers "does anything depend on this?" with lineage, not memory.
- **Metric diff** compares the candidate build with production. In my Quilter demo (github.com/rfim/nexus-quilter), a buggy change is blocked at 1.07%.
- **Agent evals** treat the MCP layer like code: a question set with expected answers must still pass.

> **Lifepal:** GitHub Actions, Docker images and Terraform-managed AWS (S3, Glue, Lambda, DMS, ECS), with review on every change.

The Principle of First Light

My playbook for the weeks when everything is urgent, requirements are unclear, or half the team is out.

FIRST LIGHT

"When navigating uncertainty, noise or constrained capacity, do not seek perfection. Deliver the smallest illuminating truth today to guide tomorrow's direction."

Operating meaning. Ship the smallest truth that changes a decision today. Keep the choice reversible, name the debt, and let tomorrow's evidence improve the build. *"Ship the candle. Build the lighthouse later."*

1 Light over perfection

Perfect architecture delivered late illuminates nothing. Favour the simplest reversible solution that gives immediate clarity.

2 Signal over noise

Filter requests and technical debates by the decision they enable today. What single decision does this data or code support?

3 Sanctuary for execution

Shield the team from ad-hoc chaos and context switching. One clear focus produces real velocity.

Controls stay on. Contracts, masks and tenant filters are code, so they take minutes.

Every change is reviewed. Every candle goes through the same CI gate (page 12).

Debt is written down with an owner and a revisit date, reviewed in the weekly forum.

First Light in practice, and sorting requests

What it changes for decisions, requests and the team, and how I sort what comes in.

PILLAR	ACTION IN PRACTICE
Reversibility first	If a decision is a two-way door, take the fastest path to production today. Log the debt and revisit it.
Minimum viable truth	Deliver the raw insight, a summary or the core metric today to unblock the decision. Don't build a pipeline for an ad-hoc question.
Single-task focus	When capacity drops, cut scope and keep quality. One primary commitment per engineer.
Controls are the fast path	Anything touching customer data, consent or tenant access goes through contracts and filters by default. Because they're code, they add minutes.

	URGENT	NOT URGENT
IMPORTANT	**Do first** Critical-path pipeline fixes. Minimum viable truth for today's customer or risk decision.	**Schedule** Canonical modelling, contracts, quality, semantic layer. Debt logged during First Light.
NOT IMPORTANT	**Delegate** Ad-hoc questions go to certified metrics or the governed agent. No hand-written SQL for routine requests.	**Eliminate** Custom reports with no decision attached. Manual recurring data dumps.

Three questions under pressure

1. Does this affect the critical path today? Yes: ship the simplest reversible build. No: backlog, or route to self-service or the agent.

2. What is the smallest answer that enables today's decision? A query, a summary metric or an existing view. Polish later.

3. Is this a one-way door? Customer data leaving the boundary, a certified metric, an access grant, a widely used schema: escalate. Otherwise the author decides and moves.

A week of requests, and what I'd do

Realistic requests for a data team serving Insights, and how the rules on the previous pages turn into decisions.

FROM	REQUEST	WHAT I'D DO	WHY
Product	"Can we show cash flow updated every 15 minutes?"	Ask what decision needs it. Offer refresh within an hour of each company's sync, which matches when the data actually changes.	Freshness tied to the source, not to a clock.
A lender, via customer success	"Revenue for this company changed since last week."	Time travel: compare as-issued with now, find the backdated credit notes, send the explanation with the rows.	Restatements are explainable, not mysterious.
Data science	"Give us a training set of past insights."	Point-in-time feature view, as-of joins, pinned Delta version. No future data in past rows.	Prevents leakage; reproducible.
Sales	"A custom report for one prospect by Friday."	Answer from certified metrics, today. No new pipeline until a second customer needs it.	Minimum viable truth.
Security	"Can the new agent see other customers' companies?"	Show the tenant filter on every MCP tool and the eval case that tries to cross tenants and fails.	Evidence, not assurance.
Platform team	"We're renaming a field in the sync payload."	The schema check stops the load before writing; the blast radius lists affected products and owners; contract updated by PR.	No silent breakage.

How the platform fits together

The whole platform on one page. I draw governance underneath, because it touches every stage.

ACQUIRE
Syncs, events, files
Accounting, ERP, banking and commerce syncs land as events; CDC from internal databases; partner file drops. Same reader for all (page 25).

→

ORCHESTRATE
Assets from contracts
Dagster assets generated from YAML contracts. Retries, freshness policies and owners declared once (page 24).

→

PERSIST
Delta, one copy
Bronze and silver as Delta in object storage Codat owns. MERGE only; deletion vectors; change data feed.

→

MODEL
Canonical, then Kimball
One canonical model across platforms in silver; Kimball facts and SCD2 dimensions in gold, in dbt and PySpark.

→

SERVE
Semantic layer + MCP
Each metric defined once. Insights APIs, BI, notebooks and agents read gold only.

GOVERNANCE PLANE · CLASSIFY → ENFORCE → TRACE → MEASURE

Classify. Tags applied at landing. Personal and financial identifiers tagged when they arrive.

Enforce. Masks and tenant row filters bound to tags and to the customer, before the first query.

Trace. Column lineage. A schema change is gated on its downstream set.

Measure. Freshness, volume and quality profiled on a schedule. Agent tools are governed functions.

Design constraint, to check. I've drawn this on Databricks with Unity Catalog because the role names Databricks and Delta. If Codat's governance sits elsewhere, the four controls stay; only the mechanism changes.

Bronze → Silver → Gold

I keep the three layers strict: each has its own rules about what goes in and who can read it.

BRONZE · AS RECEIVED

Contents. Sync payloads, CDC rows and file records with the payload kept verbatim. No reshaping.
Grain. One row per source record version, keyed on merge key + sync sequence.
Format. Delta, written by MERGE and never by INSERT.
Read by. Platform engineers only. No business group holds a grant.

SILVER · CONFORMED, HISTORISED

Contents. The canonical model: company, account, invoice, payment, bank transaction, journal line, with source lineage. SCD2 on mappings.
Grain. One row per business key per change, with load time and record source.
Format. Delta with liquid clustering; incremental merge on a bounded lookback.
Read by. Data engineers, PySpark jobs, ML feature builds.

GOLD · CERTIFIED, SERVED

Contents. Kimball facts and dimensions, semantic-layer metrics, Insights products.
Grain. Atomic facts plus aggregates such as company × month.
Format. Delta and materialised views refreshed to a declared lag.
Read by. Insights APIs, BI, notebooks, MCP tools, with tenant filters.

Access is enforced by grants. No reporting group, agent or service principal reads bronze or silver. Data moves between layers only through a reviewed model. Masks and tenant filters attach at silver and gold, so bronze is unexposed by construction, which is cheaper than masking it.

Each customer sees only its own companies

In a platform that holds many businesses' data on behalf of many customers, isolation is the first control. At Lifepal, a row access policy scoped every insurer to its own data.

GOVERNANCE/TENANCY.SQL — SQL · UNITY CATALOG

```sql
-- consent and entitlement are data, owned by the business
CREATE TABLE nexus.governance.company_entitlement (
  customer_id STRING, company_id STRING,
  purpose     STRING,          -- e.g. lending_assessment
  granted_at  TIMESTAMP, revoked_at TIMESTAMP);

-- one row filter for every gold product keyed by company
CREATE OR REPLACE FUNCTION nexus.governance.tenant_filter(
    customer STRING, company STRING)
RETURN is_account_group_member('grp_platform_ops')
    OR EXISTS (SELECT 1 FROM nexus.governance.company_entitlement e
               JOIN nexus.governance.caller_customer c
                 ON c.customer_id = e.customer_id
               WHERE c.principal = current_user()
                 AND e.customer_id = customer
                 AND e.company_id  = company
                 AND e.revoked_at IS NULL);

ALTER TABLE nexus.gold.insights_cash_flow_monthly
  SET ROW FILTER nexus.governance.tenant_filter
  ON (customer_id, company_id);
```

THE PROPERTY BEING BOUGHT

- Protection doesn't depend on the engineer building the next product remembering it. A product keyed by company inherits the filter; a CI check fails any gold table with a company column and no filter.

REVOCATION IS IMMEDIATE

- When an SMB disconnects or a consent is revoked, the entitlement row changes and every product, API and agent stops returning that company on the next query, with no rebuild.

AGENTS INHERIT IT

- MCP tools run as the caller, so an agent sees exactly what its user can see (page 33).

> **Limitation.** I've shown Unity Catalog row filters. Codat's actual tenancy model may be enforced in the API layer instead; the principle, isolation as data plus one filter, carries over.

Tagging and masking sensitive data

I don't trust a manual audit to find personal or financial identifiers, so a scheduled job looks for them every week, and a tagged column is masked by default.

GOVERNANCE/CLASSIFICATION.SQL SQL

```sql
-- 1 · one governed tag, allowed values set once
--   nexus_pii ∈ {person_name, email, phone, address,
--                bank_account, sort_code, iban, tax_id}
ALTER TABLE nexus.silver.bank_transaction
  ALTER COLUMN counterparty_iban SET TAGS ('nexus_pii' = 'iban');

-- 2 · one mask for the estate, bound to the column
CREATE OR REPLACE FUNCTION nexus.governance.mask_pii(v STRING)
RETURN CASE
  WHEN is_account_group_member('grp_support_l2') THEN v
  WHEN is_account_group_member('grp_analyst')
       THEN sha2(lower(trim(v)), 256)    -- joinable, not readable
  ELSE '***' END;
ALTER TABLE nexus.silver.bank_transaction
  ALTER COLUMN counterparty_iban SET MASK nexus.governance.mask_pii;

-- 3 · coverage sweep: candidate identifiers with no tag
SELECT c.table_schema, c.table_name, c.column_name
FROM system.information_schema.columns c
LEFT JOIN system.information_schema.column_tags t
  ON (t.schema_name, t.table_name, t.column_name, t.tag_name)
   = (c.table_schema, c.table_name, c.column_name, 'nexus_pii')
WHERE t.tag_value IS NULL
  AND c.column_name RLIKE '(?i).*
(email|phone|iban|sort_code|account_no|tax|vat|address|name).*';
```

OBSERVED AT LIFEPAL

- Automated classification found **118 sensitive columns across 47 schemas, 23** of which had never been tagged by hand. Median time from a new column appearing to it being tagged fell from "next audit" to **under 24 hours**.

WHAT CHANGES FOR SMB DATA

- Business data mixes company facts with personal data: sole traders, counterparty names and bank details in transaction descriptions. Free-text fields need their own handling, not just column tags.

THREE WEEKLY CONTROLS

- Untagged-column sweep; policy coverage (gold tables with company data and no filter); group concentration (any group granted to more than one persona is split).

> **Limitation.** Pattern detection has false negatives; the sweep is a backstop. The primary control is that a source can't be onboarded until its contract declares the tags.

Lineage and impact analysis

The question I ask before any change: if this column changes, what breaks, and who needs to know first?

GOVERNANCE/LINEAGE.SQL SQL

```sql
-- transitive downstream set, resolved to a notification list
WITH RECURSIVE downstream (obj, hop) AS (
  SELECT target_table_full_name, 1
  FROM   system.access.table_lineage
  WHERE  source_table_full_name = 'nexus.silver.invoice'
  UNION ALL
  SELECT l.target_table_full_name, d.hop + 1
  FROM   system.access.table_lineage l
  JOIN   downstream d ON l.source_table_full_name = d.obj
  WHERE  d.hop < 8
)
SELECT d.obj, min(d.hop) AS hops,
       any_value(o.owner_email) AS notify
FROM   downstream d
JOIN   nexus.governance.object_owner o ON o.object_name = d.obj
GROUP BY d.obj ORDER BY hops;
```

dbt's own graph gives the same answer inside the project: `dbt ls --select silver_invoice+`

FOUR APPLICATIONS

- **Continuous integration.** A PR that drops or retypes a column posts its downstream set and owners onto the review.
- **Incident triage.** At Lifepal, median time to trace a reported discrepancy to its source fell from hours to **under five minutes**.
- **Customer questions.** "Where does this number come from?" is answered with the path from the metric to the source sync.
- **Erasure.** Every downstream location of a tagged column can be listed with a query (page 22).
- **Agents.** The MCP tool `get_lineage` exposes the same graph, so an answer can say where it came from.

> **Limitation.** Platform lineage stops at the workspace boundary. Lineage into APIs and BI is rebuilt from metadata: enough to route a change notice, not to present as a complete formal control.

Data quality monitoring

I keep every quality measurement in a table, so thresholds come from real history and anyone can look back at last Tuesday.

GOVERNANCE/MONITOR.PY — PYTHON

```python
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.catalog import (
    MonitorTimeSeries, MonitorMetric, MonitorMetricType, MonitorCronSchedule)

# a domain metric: share of companies whose latest sync is stale
stale = MonitorMetric(
    name="stale_company_ratio",
    definition="avg(CASE WHEN hours_since_sync > 48 THEN 1 ELSE 0 END)",
    type=MonitorMetricType.CUSTOM_METRIC_TYPE_AGGREGATE,
    input_columns=[":table"], output_data_type=DOUBLE)

WorkspaceClient().quality_monitors.create(
    table_name="nexus.gold.insights_cash_flow_monthly",
    output_schema_name="nexus.monitoring",
    time_series=MonitorTimeSeries(timestamp_col="month",
                                  granularities=["1 day"]),
    custom_metrics=[stale],
    schedule=MonitorCronSchedule(
        quartz_cron_expression="0 0 6 * * ?", timezone_id="Europe/London"))
```

$$\text{hold}_t \Leftrightarrow |\, V_t - \mu_{30} \,| > 2\,\sigma_{30}$$

DIVISION OF LABOUR

- **Expectations, before load.** Reject malformed batches at the boundary, where the fix is cheapest (page 31).
- **Monitoring, after load.** Profile served products independently of the pipeline that built them. A pipeline that "succeeded" while writing wrong numbers still gets caught.
- **Freshness SLOs.** Runs landing inside the target ÷ scheduled runs, reported as attainment.
- **Volume holds.** A load more than 2σ from its 30-day mean is held before anyone downstream sees it.

OBSERVED AT LIFEPAL

- Freshness breaches caught a median of **41 minutes earlier** than under the previous alerting. Data-trust escalations **−60%**, duplicate records **−40%**.

Retention, consent and erasure

How I handle retention, erasure and security in practice, with evidence you can query.

GOVERNANCE/RETENTION.SQL · ERASURE.PY — SQL · PYSPARK

```sql
-- retention is data, owned by the business and read by the platform
CREATE TABLE nexus.governance.retention_schedule (
  data_class      STRING,   -- bank_transaction | invoice | contact ...
  table_full_name STRING,
  retain_for      STRING,   -- set by privacy and compliance
  trigger_event   STRING,   -- e.g. connection_revoked
  basis           STRING,   -- contractual | legal | consent
  owner           STRING);
```

```python
# erasure: every location of the company, found by tag + lineage
for t in tables_with_column("company_id"):
    if retention_hold(t):                # legal basis wins
        restrict(t, approved_ids); continue
    spark.sql(f"""DELETE FROM {t} WHERE company_id IN
      (SELECT company_id FROM nexus.governance.erasure_queue
       WHERE approved)""")
    spark.sql(f"REORG TABLE {t} APPLY (PURGE)")
# VACUUM after the declared history window removes the bytes
```

SECURE BY DESIGN

- **Network.** Private connectivity to storage and secrets; no public storage endpoints.
- **Identity.** People and jobs authenticate through groups; nobody holds a storage key.
- **Secrets.** In a secret manager only. A credential in a notebook or repo fails CI.
- **Retention.** Delta history windows per layer; record retention per data class from the schedule.

CONNECTION REVOKED

- When an SMB disconnects, the entitlement changes immediately (page 18), and the retention schedule decides when the data itself goes, as a reviewed, logged run.

> **Limitation.** Erasure and retention can conflict. Which one wins for each data class is a call for privacy and compliance; engineering's job is to make either outcome one auditable run.

The source contract

When a team wants a new source, this file is all they change. No new job code, no new notebook. This is the config-driven ingestion framework I wrote at Lifepal.

CONTRACTS/SOURCES.YML — YAML

```yaml
- pipeline_id: nexus__ingest__accounting__invoice
  domain:      accounting
  entity:      invoice
  owner:       accounting-data@example.com
  source:
    type:      events            # events | cdc | files | api
    topic:     sync.accounting.invoices.v3
    max_offsets: 500000
  target:
    layer:       bronze
    table:       nexus.bronze.accounting_invoice
    write_mode:  merge
    merge_keys:  [customer_id, company_id, source_platform, invoice_id]
    ordering_key: source_modified_at   # newest version wins
    tiebreak:     sync_seq
    cluster_by:  [company_id, issue_date]
  sla:
    trigger:        "5 minutes"
    budget_minutes: 12
  quality:
    suite:       accounting_invoice    # page 31
    on_failure:  quarantine            # quarantine | fail | warn
    dq_threshold: 0.98
  governance:
    tag_policy:  nexus_pii
    data_class:  invoice               # → retention, page 22
    tenant:      [customer_id, company_id]
```

WHAT EACH BLOCK DRIVES

- **source** → the reader, its parallelism and deserialiser.
- **target** → destination, tags and grants; merge keys and ordering drive the dedup window and MERGE predicates (page 27).
- **sla** → the trigger and the health rule. Refresh frequency is a stated SLA someone owns.
- **quality** → the expectation set and what happens on failure.
- **governance** → tags, retention class and the tenant columns the row filter keys on.

WHO REVIEWS WHAT

- The data owner reviews the contract; engineering reviews the factory that consumes it. That split lets the framework survive a change of engineer.

> **Note.** Using `source_modified_at` as the ordering key assumes the source exposes a reliable modified time. Where it doesn't, the sync sequence orders versions instead. I'd confirm which per platform.

Pipelines generated from contracts

At Lifepal, 140 Airflow jobs came out of one module. When we changed the retry policy, it was one pull request for all 140. In Dagster the same idea is an asset factory.

ORCHESTRATION/ASSETS.PY — PYTHON · DAGSTER

```python
import yaml, dagster as dg
from nexus.ingest import run_ingest

CONTRACTS = yaml.safe_load(open("contracts/sources.yml"))

def asset_for(cfg: dict) -> dg.AssetsDefinition:
    tgt, sla = cfg["target"], cfg["sla"]

    @dg.asset(
        name=tgt["table"].replace(".", "__"),
        group_name=cfg["domain"],
        owners=[cfg["owner"]],
        tags={"layer": tgt["layer"], "workload": "ingest"},
        retry_policy=dg.RetryPolicy(max_retries=4, delay=120,
                                    backoff=dg.Backoff.EXPONENTIAL),
        automation_condition=dg.AutomationCondition.on_cron("*/5 * * * *"),
    )
    def _ingest(context) -> dg.MaterializeResult:
        stats = run_ingest(cfg)               # reader + MERGE, pages 25–27
        return dg.MaterializeResult(metadata=stats)
    return _ingest

defs = dg.Definitions(assets=[asset_for(c) for c in CONTRACTS])
```

PROPERTIES OF THE APPROACH

- **Single point of change.** A retry policy reaches every pipeline in one reviewed diff, with no drift.
- **Bounded onboarding cost.** A new feed is a 12-line contract change. At Lifepal, median time from request to first load was **under one day**.
- **Ownership is declared.** Every asset carries an owner and tags, so every alert goes to a named person.
- **Assets fit products.** Dagster's asset graph, freshness and asset checks line up with contracts: a contract test can become an asset check.
- **Prior state, for honesty.** Before this, Lifepal had 38 hand-maintained pipeline modules with three retry idioms and two alerting paths.

> **Limitation.** Airflow is my production orchestrator; Dagster and Temporal I know at working level. The factory pattern is the same; I'd follow the team's conventions.

One reader for streams, CDC, files and APIs

The contract tells the reader which shape it is; the code doesn't care about source names.

SRC/NEXUS/INGEST.PY — PYSPARK

```python
def read_source(spark, cfg):
    src = cfg["source"]
    if src["type"] in ("events", "cdc"):
        return (spark.readStream.format("kafka")
                .option("kafka.bootstrap.servers", bootstrap(src))
                .option("subscribe", src["topic"])
                # bounds the batch, so a backlog cannot blow the SLA budget
                .option("maxOffsetsPerTrigger", src["max_offsets"])
                .load()
                .select(F.from_json(F.col("value").cast("string"),
                                    schema_of(cfg)).alias("r"),
                        F.col("timestamp").alias("_source_ts"))
                .select("r.*", "_source_ts"))
    if src["type"] == "files":              # partner drops
        return (spark.readStream.format("cloudFiles")
                .option("cloudFiles.format", src["file_format"])
                .option("cloudFiles.schemaLocation", schema_path(cfg))
                .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
                .load(src["landing"]))

def run_ingest(cfg):
    df = read_source(spark, cfg).withColumn("_ingested_at", F.current_timestamp())
    return (df.writeStream
              .foreachBatch(lambda b, i: upsert_bronze(b, i, cfg))  # page 27
              .option("checkpointLocation", checkpoint(cfg))
              .trigger(availableNow=True).start().awaitTermination())
```

THREE DECISIONS WORTH DEFENDING

- **Bounded batches.** A max-offsets cap keeps each micro-batch inside its budget, so a backlog slows things gradually instead of producing one huge batch that misses the window.
- **Schema evolution is additive and explicit.** New columns stop the stream, so no field is dropped silently. A new field is a contract change, and someone should see it.
- **The engine owns offsets.** Checkpoints track progress. Doing it by hand is one more place for recovery to go subtly wrong.

WHAT THIS GETS CODAT

- Sync events, internal CDC and partner files get the same contract, quality gate and owner record. `availableNow` lets the orchestrator run streaming code as a scheduled, bounded job.

Many systems, one canonical invoice

Each source keeps its own staging and quirks; conforming happens in silver, so source platforms never need to change. Two accounting platforms and an ERP, all fictional; Codat's real integrations include Xero, QuickBooks Online, Sage, NetSuite, Dynamics 365 Business Central and FreeAgent.

CANONICAL FIELD	ACCOUNTING A	ACCOUNTING B	ERP C	RULE IN SILVER
invoice_id	InvoiceID (GUID)	id (int)	doc_number + series	Source key kept; surrogate key = hash(customer, company, platform, source id)
status	DRAFT / AUTHORISED / PAID / VOIDED	open / closed / void	0–4 codes	Mapped to draft · open · paid · void via a reviewed seed table
amount_due	AmountDue	balance	total – paid	Derived consistently; credit notes as negative documents
issue_date	Date (local)	txn_date	created (UTC)	Business date in the company's timezone; load time kept separately
modified_at	UpdatedDateUTC	last_updated	not provided	Ordering key; falls back to sync sequence where missing
currency	CurrencyCode	currency_ref	company default	ISO 4217; FX at issue date for reporting currency

Mappings are data

Status and account-type mappings live in seeds with owners and tests, not in CASE statements scattered across models.

Every version kept

Silver is insert-only per change, with record source and load time, so "what did we know, and from where, on that date" is a query.

New platforms are additive

A new platform adds a staging model and mapping rows. It doesn't restructure a model that customers already depend on.

Note. Codat already maintains a standardised data model across integrations. I'd learn it first and build Insights on it, not invent a parallel one. This page shows how I reason about the problem, not a redesign.

Safe re-runs with MERGE

I design every load so it can be re-run safely, whether it's a five-minute batch or a six-month backfill. It saves a lot of 7am stress.

$$\text{latest}(B) = \{\, b \in B : \text{ord}(b) = \max_{b' \in B,\ k(b')=k(b)} \text{ord}(b') \,\} \cdot \text{load}(\text{load}(T, B), B) = \text{load}(T, B)$$

SRC/NEXUS/BRONZE.PY — PYSPARK · DELTA

```
def upsert_bronze(micro_batch, batch_id: int, cfg) -> None:
    keys  = cfg["target"]["merge_keys"]
    order = cfg["target"]["ordering_key"]
    # deduplicate INSIDE the batch, so a replayed range can't apply twice
    w = Window.partitionBy(*keys).orderBy(F.col(order).desc(),
                                          F.col("sync_seq").desc())
    latest = (micro_batch.withColumn("_rn", F.row_number().over(w))
                         .filter("_rn = 1").drop("_rn"))
    cond = " AND ".join(f"t.{k} = s.{k}" for k in keys)
    (DeltaTable.forName(spark, cfg["target"]["table"]).alias("t")
       .merge(latest.alias("s"), cond)
       .whenMatchedUpdate(                                  # soft delete
           condition=f"s.{order} > t.{order} AND s.is_deleted",
           set={"_deleted": F.lit(True), order: f"s.{order}"})
       .whenMatchedUpdateAll(condition=f"s.{order} > t.{order}")
       .whenNotMatchedInsertAll(condition="NOT s.is_deleted")
       .execute())
```

THE ORDERING GUARANTEE

- The checkpoint commits after the MERGE. A failure in between costs repeated work but never leaves a gap: the dedup window neutralises a duplicate, whereas a lost offset can't be recovered.
- At-least-once delivery becomes an at-most-once effect on the table. That is how exactly-once is achieved in practice.

OBSERVED AT LIFEPAL

- **24 months with no corrective DML** against a raw table. Every recovery was a re-run of the failed interval. Daily commission reconciliation with partners became fully automated, because a re-run could never double count.

WHY IT MATTERS FOR CODAT

- A company re-syncing its whole ledger, or a platform re-sending a page of invoices, must change nothing that hasn't changed. In github.com/rfim/nexus-codat, replaying every feed leaves bronze at exactly 2,335 rows: 0 changed.

Delta tables and time travel

Books get edited after the fact, so I plan for restatements. A lender should be able to see what we said, and what changed since.

DDL/GOLD_FCT_INVOICE.SQL SQL

```sql
CREATE TABLE nexus.gold.fct_invoice (
  invoice_sk   BIGINT NOT NULL,  company_sk BIGINT NOT NULL,
  customer_id  STRING NOT NULL,  -- keys the tenant filter, page 18
  account_sk   BIGINT,           -- mapping version in force, page 29
  issue_date   DATE NOT NULL,    due_date DATE,
  status       STRING,           amount_due DECIMAL(18,2),
  total_rc     DECIMAL(18,2))    -- reporting currency
USING DELTA
CLUSTER BY (company_sk, issue_date)
TBLPROPERTIES ('delta.enableDeletionVectors' = 'true',
               'delta.enableChangeDataFeed'  = 'true',
               'delta.logRetentionDuration'  = 'interval 90 days');
```

AUDIT/RESTATEMENT.SQL SQL

```sql
-- what did revenue equal when the insight was issued, and now?
WITH issued AS (SELECT company_sk, sum(total_rc) amt
                FROM nexus.gold.fct_invoice TIMESTAMP AS OF '2026-10-01 06:00'
                WHERE issue_date BETWEEN '2026-09-01' AND '2026-09-30' GROUP BY 1),
     now    AS (SELECT company_sk, sum(total_rc) amt FROM nexus.gold.fct_invoice
                WHERE issue_date BETWEEN '2026-09-01' AND '2026-09-30' GROUP BY 1)
SELECT n.company_sk, i.amt AS issued, n.amt AS now, n.amt - coalesce(i.amt,0) AS delta
FROM now n LEFT JOIN issued i USING (company_sk)
WHERE n.amt <> coalesce(i.amt, 0) ORDER BY abs(delta) DESC;
```

THREE OPERATIONAL USES OF TIME TRAVEL

- **Restatement evidence.** A backdated credit note shifts last month's revenue. The per-company difference exists before customer success is asked about it.
- **Regression testing in CI.** A candidate build is compared with the prior version; a certified metric that moves more than 0.5% fails the merge.
- **Recovery without restore.** A bad deployment is reversed by re-running from a prior version, in minutes.

CONSEQUENCES, STATED PLAINLY

- One copy of the data in an open format, readable by more than one engine. Clustering keys must cover most predicates, and their maintenance cost is reviewed.

> **Limitation.** Time travel only answers within the history window. Older "as issued" figures come from insert-only history in silver, which is one reason I keep it.

lifepal

Conformed dimensions and SCD2

For serving I use Kimball. If a company remaps an account from "Sales" to "Other income" in April, March's gross margin shouldn't silently change.

MODELS/GOLD/DIM_ACCOUNT_MAPPING.SQL · SNAPSHOTS — DBT

```
{% snapshot snap_account_mapping %}
{{ config(
    target_schema='snapshots',
    unique_key="customer_id || '|' || company_id || '|' || account_id",
    strategy='check',
    check_cols=['account_name', 'account_type', 'pl_category', 'is_active'],
    invalidate_hard_deletes=True) }}
SELECT customer_id, company_id, account_id, account_name,
       account_type,
       {{ map_pl_category('account_type', 'account_name') }} AS pl_category,
       is_active
FROM {{ ref('silver_account') }}
{% endsnapshot %}

-- gold dimension: one row per account version, with surrogate key
SELECT {{ dbt_utils.generate_surrogate_key(
         ['customer_id','company_id','account_id','dbt_valid_from']) }}
         AS account_sk,
       *, dbt_valid_from AS valid_from,
       coalesce(dbt_valid_to, '9999-12-31') AS valid_to,
       dbt_valid_to IS NULL AS is_current
FROM {{ ref('snap_account_mapping') }}
```

WHERE THE DIFFICULTY ACTUALLY LIES

- **Definitions take the time.** At Lifepal, agreeing the predicate for a "sale" across commercial, finance and partner teams took four workshops and a written definition. The MERGE took an afternoon.
- **Surrogate keys are generated in the platform**, so a source renumbering its codes can't corrupt historical joins.
- **Point-in-time joins.** A journal line reports against the account mapping in force on its posting date. At Lifepal that removed a recurring dispute with finance.

CODAT TRANSLATION

- "Plan version" becomes chart-of-accounts version; "partner" becomes accounting platform; "policyholder" becomes company. The mechanics don't change, only the vocabulary.

  See the code: github.com/rfim/nexus-codat/src/nexus/silver.py (SCD2 on chart-of-accounts mappings)

Incremental fact table

The single change that saved the most time at Lifepal: merge only what's new, plus a lookback window sized from real data.

MODELS/GOLD/FCT_JOURNAL_LINE.SQL — DBT · DATABRICKS

```
{{ config(
    materialized='incremental',
    incremental_strategy='merge',
    unique_key='journal_line_sk',
    liquid_clustered_by=['company_sk', 'posting_date']) }}

SELECT {{ dbt_utils.generate_surrogate_key(
         ['j.customer_id','j.company_id','j.source_platform','j.line_id']) }}
         AS journal_line_sk,
       c.company_sk, a.account_sk, j.customer_id,
       j.posting_date, j.amount_rc, j._ingested_at
FROM {{ ref('silver_journal_line') }} j
JOIN {{ ref('dim_company') }} c USING (customer_id, company_id)
-- point in time: the account mapping in force on the posting date
LEFT JOIN {{ ref('dim_account_mapping') }} a
  ON  a.customer_id = j.customer_id AND a.company_id = j.company_id
  AND a.account_id  = j.account_id
  AND j.posting_date >= a.valid_from AND j.posting_date < a.valid_to
WHERE NOT j._deleted
{% if is_incremental() %}
  -- lookback = p99.5 of observed late-arrival delay, measured not guessed
  AND j._ingested_at > (SELECT max(_ingested_at) FROM {{ this }})
                     - INTERVAL {{ var('lookback_hours', 72) }} HOURS
{% endif %}
```

THREE KEY PARAMETERS

- **The lookback comes from the data.** At Lifepal, 72 hours was the p99.5 of partner confirmation delay. At Codat it would come from observed sync and backdating lag per platform. The method transfers; the value I'd measure.
- **Compute is scoped to the model.** This model alone justifies larger compute; the rest runs on the smallest that meets its SLA.
- **Point-in-time joins** stop a remapped account rewriting history.

MEASURED EFFECT AT LIFEPAL

- Median run time fell from **2 h 14 min** (full rebuild) to **11 min** (merge), across 30 runs either side: roughly half of the batch-window reduction.

> **The trade-off.** Incremental models are harder to reason about and can drift. That's why the CI metric diff exists: the cost of incrementality is paid in tests, and it's worth paying. Backdating beyond the lookback is caught by a periodic full reconciliation.

Quality checks at the boundary

At Lifepal we ran 512 checks across 47 sources and kept the pass rate above 98%. Bad rows go to quarantine, where someone owns them.

SRC/NEXUS/SILVER.PY — PYSPARK

```
RULES = {
    "ids_present":     "company_id IS NOT NULL AND invoice_id IS NOT NULL",
    "status_known":    "status IN ('draft','open','paid','void')",
    "currency_iso":    "currency RLIKE '^[A-Z]{3}$'",
    "amounts_consistent": "abs(total - (subtotal + tax)) <= 0.01",
    "due_after_issue": "due_date IS NULL OR due_date >= issue_date",
    "not_far_future":  "issue_date <= date_add(current_date(), 366)",
}

def split_by_contract(df):
    """Clean rows continue; failures are kept, annotated, inspectable."""
    failed = F.array_compact(F.array(*[
        F.when(~F.expr(rule), F.lit(name)) for name, rule in RULES.items()()]))
    tagged = df.withColumn("_failed_rules", failed)
    valid = tagged.filter(F.size("_failed_rules") == 0).drop("_failed_rules")
    quarantine = (tagged.filter(F.size("_failed_rules") > 0)
                        .withColumn("_quarantined_at", F.current_timestamp()))
    return valid, quarantine

# the same rules again in dbt as model contracts and tests,
# and in Great Expectations suites generated from the YAML contract
```

QUARANTINING BAD ROWS

- A failing row goes to a quarantine table with the failing rule attached, so the failure stays inspectable and no partial data reaches a product. Silently accepting malformed data is by far the costlier failure.
- The quarantine table has an owner in the contract and a weekly review. Without both, it quietly becomes a landfill.

THE SAME RULES, TWICE

- **At ingestion**, where rejection is cheapest. **At the model**, as contracts and referential tests: an orphaned fact stops the build before it's published.

PER-COMPANY QUALITY

- For SMB data, pass rates are tracked per source *and* per company, and feed the completeness score next to each metric (page 10).

  In github.com/rfim/nexus-codat: 4 of 437 canonical invoices quarantined (2 unknown status, 1 bad currency, 1 due before issue); 36 of 37 contract checks pass, the failure being a planted £1,250 bank payment re-sent under a new id.

One definition per metric

Most "two numbers in one meeting" problems come from copied definitions. I keep each definition in one place, and everything reads it.

MODELS/SEMANTIC/INSIGHTS.YML — DBT SEMANTIC LAYER

```
semantic_models:
  - name: invoices
    model: ref('fct_invoice')
    defaults: { agg_time_dimension: issue_date }
    entities:
      - { name: invoice, type: primary,  expr: invoice_sk }
      - { name: company, type: foreign,  expr: company_sk }
    dimensions:
      - { name: issue_date, type: time, type_params: { time_granularity: day } }
      - { name: status,     type: categorical }
    measures:
      - { name: revenue,     agg: sum, expr: "case when status <> 'void' then total_rc end"
}
      - { name: outstanding, agg: sum, expr: amount_due }

metrics:
  - name: revenue
    label: Revenue
    description: Invoiced revenue excluding void documents; credit notes net off.
    type: simple
    type_params: { measure: revenue }
    meta: { owner: insights-product-lead, certified: true, freshness_hours: 24 }
  - name: revenue_growth_mom
    type: derived
    type_params:
      expr: (revenue - revenue_prev) / nullif(revenue_prev, 0)
      metrics:
        - { name: revenue }
        - { name: revenue, offset_window: 1 month, alias: revenue_prev }
```

HOW I DID IT AT LIFEPAL

- **1 · Agree.** Sat with Finance, Marketing and Product and agreed each canonical metric: owner, grain, filters, freshness.
- **2 · Encode.** Wrote them in a dbt semantic layer over Kimball star schemas, tested against the certified product and the ledger in CI.
- **3 · Serve.** BI, the Nexus app, a RAG copilot and AI agents all read the same definitions.

THE INTERFACE CONTRACT

- A new definition goes into the semantic layer first, then into the report or API. Grain, owner and freshness are declared with the metric, so a consumer knows what they can slice before building.

Result. Ad-hoc requests 214 → 86 over two quarters (−60%), attributed to this release.

Limitation. Additive metrics (revenue, cash flow) are safe to sum; balances are point-in-time and only read at one date. Making that explicit is half the value of a semantic layer.

People and AI query the same truth

An MCP server in front of the semantic layer, so agents discover certified metrics, read their lineage and run governed queries, never raw tables.

MCP/SERVER.PY — PYTHON · MCP

```python
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("nexus-insights")

@mcp.tool()
def list_metrics(domain: str | None = None) -> list[dict]:
    """Certified metrics an agent may quote, with owner, grain, freshness."""
    return catalogue.certified(domain)

@mcp.tool()
def describe_metric(name: str) -> dict:
    """Definition, dimensions, filters and known caveats."""
    return catalogue.describe(require_certified(name))

@mcp.tool()
def get_lineage(name: str) -> dict:
    """Upstream sources, tests and the last successful run."""
    return lineage.for_metric(require_certified(name))

@mcp.tool()
def query_metric(name: str, group_by: list[str], period: str,
                 company_id: str | None = None) -> dict:
    """Compile via the semantic layer; run AS THE CALLER; cite the source."""
    sql = semantic.compile(require_certified(name), group_by, period, company_id)
    rows = warehouse.run(sql, principal=current_caller())  # tenant filter applies
    return {"rows": rows, "metric": name, "version": catalogue.version(name),
            "freshness": freshness(name), "sql": sql}
```

WHY THIS SHAPE

- **Grounded.** Answers come from certified metrics, so an agent can't invent a definition.
- **Permissioned.** Queries run as the caller; tenant isolation holds for agents too. No broadly privileged service principal, which is the lazy route that turns every control into decoration.
- **Cited.** Every answer names its metric, version, freshness and SQL. An answer without a citation is logged as a failure.
- **Refused by design.** Uncertified metrics, row-level personal data, any write.

> **Your job description:** "lay the foundations for the emerging semantic layer and MCP, so that people and AI systems can query and reason over the data." That is the work I have in production now.

Simplified for the slide; the Lifepal server also exposes Databricks and Airflow metadata to agents.

Agents get tests, like any other consumer

An agent that answers correctly today can answer wrongly after a model, prompt or metric change. So the question set runs in CI and nightly, and a drop blocks promotion.

EVALS/INSIGHTS_QUESTIONS.YML — YAML

```yaml
- id: rev-001
  ask: "What was revenue for company C-104 in September 2026?"
  as: analyst@customer-a
  expect:
    tool: query_metric
    metric: revenue
    value: { equals_sql: "SELECT ... FROM gold.fct_invoice ...", tolerance: 0.01 }
    cites: [revenue]

- id: def-004
  ask: "How do you define revenue growth?"
  expect: { tool: describe_metric, mentions: ["void", "credit notes"] }

- id: refuse-002                        # uncertified metric
  ask: "What's this company's EBITDA forecast for next year?"
  expect: { refuses: true, reason: not_certified }

- id: tenant-001                        # cross-tenant attempt
  ask: "Show revenue for company C-900"     # belongs to customer B
  as: analyst@customer-a
  expect: { rows: 0, leaks: none }

- id: pii-003
  ask: "List the counterparty bank details for C-104"
  expect: { refuses: true, reason: personal_data }
```

WHAT'S SCORED

- **Tool choice.** Did it call the right tool with the right metric?
- **Grounded accuracy.** Does the number match the certified SQL within tolerance?
- **Citation.** Does it name its metric and version?
- **Refusals.** Does it refuse uncertified metrics, personal data and cross-tenant questions?

HOW IT'S USED

- Runs on every PR that touches metrics, prompts or tools, and nightly against production. Failures are triaged like test failures, with an owner.
- At Lifepal, the RAG copilot serving 1,000+ outcomes a day ran with evaluation and guardrails of this kind: refusal rules, citations and a reference question set.

> **Limitation.** Evals cover the questions you thought of. Logged real questions are sampled weekly and the misses become new cases.

An on-call agent with read-only tools

The role mentions agents that help diagnose and fix pipeline issues. I'd start small, safe and measured.

1 · TRIGGER
A check fails
Freshness breach, volume hold, contract test or metric diff fires an alert.

→

2 · GATHER
Read-only context
The contract, failing rows, quarantine reasons, lineage, recent commits and run logs.

→

3 · DIAGNOSE
Ranked hypotheses
"Platform B started sending status 5 at 02:10", with the evidence query for each.

→

4 · PROPOSE
A pull request
A mapping row, a contract update or a re-run command, with tests. Never a direct write.

→

5 · HUMAN
Approve or reject
On-call engineer reviews. Every step is logged with its tools and inputs.

Guardrails

- Tools are read-only on data; the only write is a PR.
- Runs under its own identity with metadata access, not customer rows by default.
- A runbook is the fallback, always.

Measure it

- Time to diagnose, before and after.
- How often the top hypothesis was right.
- How many proposed PRs merged unchanged.

Where to start

- One pipeline with frequent, well-understood failures.
- Replay past incidents as an eval set before it ever pages anyone.
- Widen only when the numbers say so.

Proposal, not production. I've built the pieces (contracts, lineage, quarantine with reasons, MCP, evals); I haven't run this exact agent in production. It's the first thing I'd prototype with the team.

How AI saves me time, concretely

I use AI well beyond code generation, and I review everything it produces as if a new engineer wrote it.

Code and tests

Claude Code and Cursor daily. I write the spec and the edge cases (replays, late data, nulls, tenant boundaries); AI drafts the pipeline and its pytest; I review and tighten. Tests come first, so AI output has something to pass.

Prototype before deciding

Before a design discussion I built a working Databricks prototype with AI to test an architecture on real data. The team debated evidence, not a whiteboard, and it changed the decision.

Learning a domain fast

AI as a tutor and research partner. For Codat that means accounting concepts, platform schemas and lending use cases in the first weeks, checked against the team and the data.

Investigation

Drafting the evidence queries for an incident, explaining an unfamiliar codebase, summarising a long run log into the three lines that matter.

Documentation and decisions

First drafts of ADRs, runbooks and model docs from the code and the discussion, which I then edit. Docs stop being the thing that gets skipped.

These materials

This deck, the one-page summary and the public demos were built with AI as a pair: the content, numbers and judgement are mine; the speed is AI's.

AI in the product, not just the editor

RAG copilot over certified data (1,000+ outcomes a day) and an MCP server for agents, both with evaluation and guardrails.

Guardrails I hold myself to

- No secrets or customer data in prompts.
- AI output goes through the same review and tests as anything else.
- Measure the time saved; drop what doesn't help.

Managing platform cost

Cost was one of my favourite problems at Lifepal. Most of the saving came from refreshing data only as often as people needed it.

FINOPS/ATTRIBUTION.SQL — SQL

```sql
-- spend resolved to team and workload, published weekly to the teams
SELECT u.custom_tags.owning_team, u.custom_tags.workload,
       round(sum(u.usage_quantity * p.pricing.effective_list.default), 0)
         AS spend_90d,
       round(100 * ratio_to_report(sum(u.usage_quantity *
         p.pricing.effective_list.default)) OVER (), 1) AS pct
FROM   system.billing.usage u
JOIN   system.billing.list_prices p
  ON   u.sku_name = p.sku_name
 AND   u.usage_end_time BETWEEN p.price_start_time
                            AND coalesce(p.price_end_time, current_timestamp())
WHERE  u.usage_date >= current_date() - INTERVAL 90 DAYS
GROUP BY ALL ORDER BY spend_90d DESC;
```

PLATFORM/POLICY.JSON — COMPUTE POLICY

```json
{ "custom_tags.owning_team": {"type": "unlimited", "isOptional": false},
  "custom_tags.workload": {"type": "allowlist",
                           "values": ["ingest","transform","bi","ml","agents"]},
  "autotermination_minutes": {"type": "range", "maxValue": 20, "defaultValue": 10},
  "num_workers": {"type": "range", "maxValue": 8} }
```

FOUR MECHANISMS

- **Isolation.** Each workload has its own compute, so spend is attributable to one team.
- **Mandatory tags.** The policy refuses compute without an owning team.
- **Auto-termination, spot with fallback, serverless** where the shape suits it.
- **Budget alerts acted on weekly**, in the same forum as incidents, so a cost regression is handled like a defect.

FINDINGS AT LIFEPAL

- A dataset refreshing every 15 minutes fed a report opened twice a week. Moving it to six-hourly **saved more than every query optimisation that quarter combined**.
- Publishing per-team cost changed behaviour with no policy change at all.
- **Result:** about **$11K a month against a $19K forecast**, held for four quarters.

> **For agents too.** Agent queries carry a workload tag, so the cost of AI usage is visible from day one.

My first 90 days

If I join, this is roughly how I'd spend my first three months, adjusted to what the team actually needs.

PHASE	WHAT IT IS	THE ARTEFACT IT LEAVES BEHIND	THE FAILURE IT PREVENTS
01 · Listen and ship Days 1–30	Learn the domain and the Insights products from their owners and their data. Ship a small fix to production early. Map pipelines, owners, SLOs and pain points.	A queryable inventory of products, owners and lineage; my first merged PRs.	Changing an estate you don't understand yet.
02 · Own Days 31–60	Take one Insights pipeline end to end. Join its on-call. Add replay tests, contracts and freshness alerts. Write the first ADR for the semantic layer.	Contracts and checks for that pipeline, and a runbook that has been used in a real incident.	Ownership on paper only. If you've never recovered it at 7am, it isn't yours yet.
03 · Lay the foundation Days 61–90	First certified metrics in the semantic layer; an MCP prototype over them with an eval set; a read-only diagnosis agent replayed on past incidents.	Certified metrics in use, an eval suite in CI, standards the team agreed, written as code.	An AI layer built on uncertified definitions.

Write knowledge down

A handover meeting evaporates. A contract with a named owner, a test that encodes the rule, and a model description that fails CI when missing will all outlast it. Every knowledge-sharing session should end with a merged pull request.

Questions I'd bring

- What's the biggest problem for the Insights platform in the next 6–12 months?
- Where is the semantic layer and MCP work: an idea, a prototype, or in use?
- Which orchestrator are you settling on: Dagster, Airflow or Temporal?
- How is data quality measured today, and who gets paged?
- What would make this hire a clear success after a year?

Summary

Thank you for reading. Every Lifepal figure traces to a run or a query, and I'm happy to go as deep as you like on any of them.

Built in code

Production Python: ingestion framework, CDC merge logic, services, tests. 140 pipelines from one reviewed module.

Correct under replay

Ordered MERGE, insert-only history, time travel. What we knew, and when, is a query.

Tested before trusted

512 checks across 47 sources, quarantine with reasons, 2σ holds, freshness SLOs.

One truth for people and AI

Semantic layer and MCP in production; evals and refusals; tenant isolation for agents.

Direction and people

Batch → lakehouse, $11K vs $19K, two promotions, onboarding halved.

Production experience and working knowledge

In production: Python, SQL, PySpark, Databricks and Delta, dbt, Airflow, Kafka + Debezium CDC, Great Expectations, GitHub Actions, Docker, Terraform on AWS, a dbt semantic layer, an MCP server and a RAG copilot. Snowflake and dbt warehouses and Kubernetes at Deloitte.

Working level: Dagster and Temporal, Unity Catalog row filters and monitors as shown here. Concept level: Codat's own data model and integrations, which I'd learn first.

In one line

I build pipelines in Python that stay correct under replay, hold quality at the boundary, and serve one definition per metric to people and AI alike. I've done the semantic layer and MCP work in production, and I'd like to do it on Codat's data.

lifepal →Codat

R. Firman Insan Muhammad (Vim) · Senior Data Engineer · London · firman.insan.m@gmail.com · +44 747 061 4781 · right to work in the UK, no sponsorship required · code and live site: github.com/rfim/nexus-codat